

【靠谱的车】

题目描述

程序员 小明打了一辆出租车去上班。出于职业敏感，他注意到这辆出租车的计费表有点问题，总是偏大。

出租车司机解释说他不喜欢数字4，所以改装了计费表，任何数字位置遇到数字4就直接跳过，其余功能都正常。

比如：

1. 23再多一块钱就变为25；
2. 39再多一块钱变为50；
3. 399再多一块钱变为500；

小明识破了司机的伎俩，准备利用自己的学识打败司机的阴谋。

给出计费表的表面读数，返回实际产生的费用。

输入描述

只有一行，数字N，表示里程表的读数。

($1 \leq N \leq 888888888$)。

输出描述

一个数字，表示实际产生的费用。以回车结束。

用例

输入	5
----	---

输出	4
说明	5表示计费表的表面读数。 4表示实际产生的费用其实只有4块钱。

输入	17
输出	15
说明	17表示计费表的表面读数。 15表示实际产生的费用其实只有15块钱。

输入	100
输出	81
说明	100表示计费表的表面读数。 81表示实际产生的费用其实只有81块钱。

题目解析

看到题目提示输入数字N的取值范围 $1 \leq N \leq 888888888$ ，我陷入了沉思，这是要我们设计一个 $O(1)$ **时间复杂度** 的算法呀，因为就算是 $O(n)$ 也遭不住9个8的数量级啊。

原型题应该是：[算法 - 里程表故障_某辆车的里程表出现了故障:它总是跳过数字3和数字8。](#)也就是说,当前显示已走过两公-CSDN博客

本题的解题思路类似，但是不是 **八进制** ，而是九进制

						实际费用 计费表费用					
						1		1	1×9^0		
						2		2	2×9^0		
						3		3	3×9^0		
						4		5	$(5-1) \times 9^0$		
						5		6	$(6-1) \times 9^0$		
						6		7	$(7-1) \times 9^0$		
						7		8	$(8-1) \times 9^0$		
						8		9	$(9-1) \times 9^0$		
					逢9进1	9		10	$1 \times 9^1 + 0 \times 9^0$		
						10		11	$1 \times 9^1 + 1 \times 9^0$		
						11		12	$1 \times 9^1 + 2 \times 9^0$		
						12		13	$1 \times 9^1 + 3 \times 9^0$		
计费表工作原理相当于九进制计算器						13		15	$1 \times 9^1 + (5-1) \times 9^0$		
						14		16	$1 \times 9^1 + (6-1) \times 9^0$		
						15		17	$1 \times 9^1 + (7-1) \times 9^0$		
						16		18	$1 \times 9^1 + (8-1) \times 9^0$		
						17		19	$1 \times 9^1 + (9-1) \times 9^0$		
						18		20	$2 \times 9^1 + 0 \times 9^0$		

实际费用 十进制	计费表费用 十进制	实际费用 九进制	
		9^1	9^0
1	1	0	1
2	2	0	2
3	3	0	3
4	5	0	5-1
5	6	0	6-1
6	7	0	7-1
7	8	0	8-1
8	9	0	9-1
9	10	1	0
10	11	1	1
11	12	1	2
12	13	1	3
13	15	1	5-1
14	16	1	6-1
15	17	1	7-1
16	18	1	8-1
17	19	1	9-1
18	20	2	0

CSDN @伏城之外

也就是说：

比如,

计费表费用是13的话, 那么对应的真实费用是 $1 * 9^1 + 3 * 9^0 = 12$

计费表费用是15的话, 那么对应真实费用就是 $1 * 9^1 + (5-1) * 9^0 = 13$

Java算法源码

```
1 import java.util.Arrays;
2 import java.util.Scanner;
3
4 public class Main {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7
8         int[] arr = Arrays.stream(sc.nextLine().split("")).mapToInt(Integer::parseInt).toArray();
9
10        System.out.println(getResult(arr));
11    }
12
13    public static int getResult(int[] arr) {
14        int correct = 0;
15
16        for (int i = 0; i < arr.length; i++) {
17            int fault = arr[i];
18            if (fault > 4) fault--;
19
20            for (int j = arr.length - i - 1; j > 0; j--) {
21                fault *= 9;
22            }
23
24            correct += fault;
25        }
26
27        return correct;
28    }
```

JS算法源码

```
1  /* JavaScript Node ACM模式 控制台输入获取 */
2  const readline = require("readline");
3
4  const rl = readline.createInterface({
5    input: process.stdin,
6    output: process.stdout,
7  });
8
9  rl.on("line", (line) => {
10    const arr = line.split("").map((ele) => parseInt(ele));
11
12    let correct = 0;
13    for (let i = 0; i < arr.length; i++) {
14      // 遍历输入数的每一位
15      let fault = arr[i];
16      if (fault > 4) {
17        // 如果本位数比4大，则相当于跳过1次，则需要将本位数-1
18        fault--;
19      }
20
21      for (let j = arr.length - i - 1; j > 0; j--) {
22        // 将本位转成十进制
23        fault *= 9;
24      }
25
26      correct += fault; // 累加各位对应十进制数
27    }
28    console.log(correct);
29  });
```

[运行](#)

Python算法源码

```
1 # 输入获取
2 arr = list(map(int, list(input()))))
3
4
5 # 算法入口
6 def getResult():
7     correct = 0
8     for i in range(len(arr)):
9         fault = arr[i]
10         if fault > 4:
11             fault -= 1
12
13         for j in range(len(arr) - i - 1, 0, -1):
14             fault *= 9
15
16         correct += fault
17     return correct
18
19
20 # 调用算法
21 print(getResult())
```

C算法源码

```
1 #include <stdio.h>
2
3 int getResult(const int nums[], int nums_size);
4
5 int main() {
6     int n;
7     scanf("%d", &n);
```

运行

```
8 | 9 |     int nums[10];
10 |    int nums_size = 0;
11 |
12 |    while (n > 0) {
13 |        nums[nums_size++] = n % 10;
14 |        n /= 10;
15 |    }
16 |
17 |    int l = 0;
18 |    int r = nums_size - 1;
19 |
20 |    while (l < r) {
21 |        int tmp = nums[l];
22 |        nums[l] = nums[r];
23 |        nums[r] = tmp;
24 |
25 |        l++;
26 |        r--;
27 |    }
28 |
29 |    printf("%d\n", getResult(nums, nums_size));
30 |
31 |    return 0;
32 | }
33 |
34 | int getResult(const int nums[], int nums_size) {
35 |     int correct = 0;
36 |
37 |     for (int i = 0; i < nums_size; i++) {
38 |         int fault = nums[i];
39 |         if (fault > 4) fault--;
40 |
41 |         for (int j = nums_size - i - 1; j > 0; j--) {
42 |             fault *= 9;
43 |         }
44 |
45 |         correct += fault;
```



```
46 |     }
    |     47 |
48 |     return correct;
49 | }
50 |
51 |
```

C++ 算法源码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      string s;
6      cin >> s;
7
8      vector<int> nums;
9      for (const auto &c: s) {
10         nums.emplace_back(c - '0');
11     }
12
13     auto len = nums.size();
14
15     int correct = 0;
16
17     for (int i = 0; i < len; i++) {
18         int fault = nums[i];
19         if (fault > 4) fault--;
20
21         for (auto j = len - i - 1; j > 0; j--) {
22             fault *= 9;
23         }
24
25         correct += fault;
```

```
26 |     }  
27 |  
28 |     cout << correct << endl;  
29 |  
30 |     return 0;  
31 | }
```